Dynamic AI Firewall: Frontend Specification (SaaS Console)

Version 1.0 · Prepared for Abraham · 24 October 2026 · Status: Draft for review

1. Purpose and scope

This document specifies the frontend of the Dynamic AI Firewall SaaS platform: the web console used by customers (banks, fintech, government, telecom) and the separate operator console used by the platform team. It covers users, technology choices, architecture, screens, safety and explainability UX, security requirements, SaaS commercial features, testing, deployment and a phased roadmap.

It does not cover the sensor agent, the detection and RL engine, or backend internals, except where the frontend depends on them (API contracts, real-time streams, audit data).

Guiding principle: the console is where humans stay in control of an AI system that can affect production banking and government traffic. Every design decision favors clarity, explainability, reversibility and tenant isolation over visual flash.

2. Product context and assumptions

- The product is a **multi-tenant SaaS**. Customers install a lightweight sensor in their network; the cloud platform handles training, threat intelligence, fleet management, dashboards and reporting.
- The cloud is **not in the packet path**. Enforcement decisions are made at the sensor; the console manages, reviews and approves.
- AI actions follow a **tiered autonomy model**: monitor only, recommend, supervised auto (human approval for destructive actions), and full auto per action class.
- New tenants start in **detection-only mode**.
- Target buyers have long procurement cycles and will ask for SSO, audit evidence, a trust center and data-residency guarantees.
- Languages: English first, Amharic planned (to be confirmed, see section 18).
- Exact library versions must be verified and pinned at project start; this document names tools, not versions.

3. Users and roles

Role	Primary goals	Access
SOC Analyst	Triage alerts, investigate flows, read explanations, approve or reject AI actions	Alerts, incidents, approvals, live feed
Security / IT Manager	Posture, trends, compliance evidence, policy and autonomy settings	Dashboards, reports, policy
Tenant Admin	Users, SSO, API keys, integrations, sensors, billing	Everything in the tenant
Auditor (read-only)	Evidence for regulators and internal audit	Audit log, reports
Platform Operator (internal)	Tenant health, fleet, model rollout, support	Separate operator console only

Permissions are enforced by the backend on every request. The frontend only hides or disables what a user cannot do; it is never the control.

4. Technology stack

Area	Choice	Reason
Framework	Next.js (App Router) with TypeScript	One codebase for marketing site, docs, signup and tenant console; SSR for public pages
Styling and components	Tailwind CSS, shadcn/ui (Radix primitives)	Accessible components owned in our repo
Component workshop	Storybook	Build and review UI in isolation
Server state	TanStack Query	Caching, retries, pagination
Client state	Zustand	Filters, selection, UI preferences

Area	Choice	Reason
Tables	TanStack Table with virtualization	Alert and flow tables can reach millions of rows
Charts	Apache ECharts	Strong large time-series performance
Maps	MapLibre GL	Open source, no per-load fees
Graph view	Cytoscape.js or React Flow for v1; deck.gl or Sigma.js (WebGL) at scale	Attack paths and topology
Real time	WebSocket for alert streams, SSE for simple one-way feeds	With batching and backpressure
Forms	React Hook Form with Zod	Validation shared with API types
API client	Generated from OpenAPI (for example openapi-typescript)	Frontend stays in sync with FastAPI
Authentication	OIDC provider with SSO (SAML/OIDC), SCIM, MFA and passkeys	See section 9 on provider and data residency
Billing	Stripe Billing and Customer Portal	Subscriptions, invoices, usage metering
Feature flags	Unleash or LaunchDarkly	Per-tenant and per-plan rollout
Internationalization	next-intl	English and Amharic
Observability	OpenTelemetry, Sentry, Grafana	Trace slow screens, catch errors
Testing	Vitest, React Testing Library, Playwright	Unit, component, end-to-end
Mocking	MSW (Mock Service Worker)	Build UI before backend is ready
Hosting	Kubernetes or a managed platform behind CDN and WAF, regional deployments	Data-region control

5. Architecture

5.1 Applications

1. **Public site**: marketing, docs, pricing, trust center, status page (SSR/static).
2. **Tenant console**: the authenticated product (section 6).
3. **Operator console**: internal, separate deployment and separate identity realm, stricter access controls (section 11).

5.2 Tenant resolution and routing

- Each tenant gets a subdomain (for example `bank-a.example.com`).
- The tenant is resolved from the **authenticated server session**, never from a client-supplied parameter or header.
- All API calls carry the session; the backend binds every query to the tenant in the token.
- Regional deployments serve tenants pinned to a data region; the console shows the tenant's data region in the header.

5.3 Data flow

- Browser to Next.js server (session, SSR for shell) and to the public API gateway for data.
- Typed API client generated from the OpenAPI specification; all responses validated with Zod at the boundary.
- Real-time alerts arrive over an authenticated WebSocket per tenant; the client batches updates (for example every 250 ms), virtualizes lists and caps in-memory history.
- If the stream drops, a persistent banner states that data may be stale and shows the last update time. Stale data is never shown silently.

5.4 Error and empty states

Every screen defines loading, empty, partial-data, error and permission-denied states. In a security product, an empty screen must say why it is empty (no alerts, sensor offline, filter too narrow, or still waiting for data).

6. Tenant console: screens and features (v1)

1. **Overview.** Threat level, alerts by severity, blocked versus flagged, sensor health, mean time to triage, current autonomy mode. A few actionable widgets, no decoration.
2. **Alert queue (core screen).** Sortable, filterable, grouped by incident. Columns: severity, MITRE ATT&CK technique, confidence, novelty score, source and destination, sensor, status, assignee. Saved filters, bulk actions, keyboard navigation.
3. **Incident detail.** Timeline of events, the flows involved, assignee and status, notes, and the explainability panel (section 7). Actions: approve, reject, escalate, add to allowlist, mark false positive, with impact preview.
4. **Approvals.** Pending AI-recommended actions (block, quarantine, honeypot redirect) awaiting human sign-off, with expiry, risk level and one-click reject. This screen implements the tiered autonomy model.
5. **Live feed.** Real-time stream with pause, filter and rate indicator. Not the home page.
6. **Network and asset map.** Topology, sensor locations, lateral-movement paths with ATT&CK stage overlays.
7. **Policy.** Static rules and AI-generated rules; per-action-class autonomy level; allowlists; schedules; global **kill switch** to force monitor-only mode.
8. **Sensors and models.** Sensor status and version, enrollment, active model version, shadow and canary status, rollback request.
9. **Reports and compliance.** Evidence exports for PCI DSS, ISO 27001 and relevant national banking requirements (PDF and CSV), scheduled delivery. Regulatory references must be verified against primary sources before release.
10. **Audit log.** Immutable, searchable record of every human and AI action with actor attribution, exportable.
11. **Settings.** Users and roles, SSO and SCIM, API keys, webhooks, SIEM integrations (Splunk, QRadar and others), notification channels, branding, language and timezone.
12. **Billing and plan.** Plan, usage (sensors, flows per day), invoices, payment method, contract details. Enterprise tenants pay by invoice and purchase order as well as card.

7. Explainability and safety UX

7.1 Explainability panel (required on every AI decision)

- Plain-language primary reason and supporting evidence.
- Top contributing features (from the explainability service).

- Confidence and novelty shown separately, with honest uncertainty. A "novel" flow is unusual, not necessarily malicious, and the wording must say so.
- MITRE ATT&CK mapping where available.
- Similar past events and their outcomes.
- Model version that made the decision.

7.2 Autonomy levels

Level	Behavior
Monitor only	Detect and alert, no enforcement (default for new tenants)
Recommend	AI proposes actions, humans execute
Supervised auto	Low-risk actions automatic, destructive actions need approval
Full auto	Per action class, opt-in, with limits

Levels are set per action class (for example rate limit versus quarantine). Raising a level requires re-authentication and is written to the audit log.

7.3 Dangerous-action rules

- Confirmation shows an impact preview (for example "this will block 14 active sessions").
- Every automatic block has an expiry and a visible undo.
- Quarantine of internal hosts always requires human approval in v1.
- The kill switch is always visible to authorized roles and takes effect with a clear confirmation of state.
- AI versus human attribution appears on every action in every history view.

7.4 Alert fatigue controls

Grouping of related alerts, deduplication, suppression rules with expiry, default "needs action" view, and a metric for analyst load.

8. Onboarding and sensor enrollment

Activation decides whether pilots convert, so onboarding is a designed flow:

1. Invite or signup, email verification, MFA enrollment.
2. Organization setup wizard: name, region, timezone, language, roles.
3. SSO configuration (optional at first, required for enterprise plans).
4. **Sensor enrollment:** generate a one-time token, show the install command, live status "waiting for first data", then confirmation with sensor health.
5. Baseline learning period explained on screen, with progress and what the product will and will not do during it.
6. Detection-only mode confirmed, with a guided path to raise autonomy later.
7. Integrations (SIEM, webhooks, notifications) and a first test alert.

9. Security requirements

- **Sessions:** httpOnly, Secure, SameSite cookies. No tokens in localStorage. Short session lifetime with idle timeout.
- **Step-up authentication:** re-authentication for approving quarantine, raising autonomy, changing SSO, creating API keys.
- **MFA:** enforced; passkeys/WebAuthn supported and recommended for approvers.
- **Identity provider:** a managed OIDC provider is acceptable for early pilots and commercial tenants. For government tenants with residency requirements, identity data must stay in the tenant's region, which implies a region-pinned or self-hosted provider (for example Keycloak). The provider choice must be confirmed per tenant class.
- **Hostile data:** log, flow and payload content is attacker-controlled. It is always escaped, never rendered as HTML, and never used in `dangerouslySetInnerHTML`. Long or malformed strings are truncated safely.
- **Content Security Policy:** strict, no inline scripts, restricted third-party origins, Subresource Integrity on any external asset.
- **CSRF protection** on state-changing requests; origin checks on WebSocket connections.
- **Tenant isolation:** automated tests prove one tenant cannot read another's data through UI routes, API calls, cached responses or WebSocket channels.

- **Supply chain:** lockfiles, dependency scanning, signed builds, SBOM generation, minimal third-party runtime dependencies.
- **Support access:** staff access to a tenant is time-limited, requires customer consent or a ticket, and is fully audit-logged.
- **Privacy:** no third-party analytics that receive tenant data; product analytics are first-party and exclude traffic content.

10. SaaS commercial and trust features

- Plans, usage metering and limits surfaced in the UI; upgrade prompts that never block security-critical functions.
- Team management, role assignment, SSO, SCIM provisioning, API keys with scopes and rotation.
- Webhooks and SIEM integrations with delivery status and retry visibility.
- **Trust center:** security overview, compliance documents, penetration test summaries, sub-processor list, data-region information. Only claims that are true and evidenced are published.
- **Status page** and in-app incident banners; public changelog.
- In-app help, documentation search and a support request flow that attaches diagnostic context with customer consent.

11. Operator console (internal)

A separate application and identity realm with stricter controls (hardware-key MFA, IP allow-listing, full audit):

- Tenant list, health and usage.
- Sensor fleet status and versions.
- Model rollout management per tenant: shadow, canary, promote, roll back.
- Support impersonation with consent, time limit and audit trail.
- Billing and usage overview.
- Feature flag management.

The operator console must never share sessions, cookies or deployment with the tenant console.

12. Non-functional requirements

Area	Requirement
Performance	Alert queue interactive in under 2 seconds on a typical analyst laptop with 100,000 loaded rows (virtualized); live feed handles bursts without freezing the UI
Availability	Console availability target aligned with the platform SLA; the console failing must never affect enforcement at sensors
Accessibility	WCAG 2.2 AA; full keyboard operation of triage flows; color is never the only severity signal
Internationalization	All strings externalized; English and Amharic; locale-aware dates and numbers; timezone per user
Browsers	Current and previous major versions of Chrome, Edge, Firefox and Safari
Theming	Dark mode default, light mode available, high-density table option
Responsiveness	Desktop first; tablet usable for alert review; mobile limited to notifications and read-only views in v1
Resilience	Graceful degradation when APIs or streams fail, with clear user messaging

13. Testing and quality

- **Unit and component tests:** Vitest and React Testing Library, with Storybook stories for every shared component.
- **End-to-end:** Playwright covering login, MFA, onboarding, alert triage, approvals, autonomy changes, kill switch and report export.
- **Tenant isolation tests:** run in CI against the API and UI with two seeded tenants.
- **Security tests:** XSS payloads in flow and log fields, CSRF, session handling, dependency scanning, periodic third-party penetration test.

- **Accessibility tests:** automated checks in CI plus manual keyboard and screen-reader review before each release.
- **Contract tests:** the OpenAPI schema is the source of truth; breaking changes fail the build.
- **Performance tests:** large-dataset fixtures for tables, graphs and the live feed.

14. Observability and deployment

- Environments: local (MSW mocks), development, staging, production, with a production-like staging that includes seeded multi-tenant data.
- CI/CD: lint, type-check, tests, build, container scan, deploy to staging, automated end-to-end run, manual promotion to production.
- Release safety: feature flags, canary rollout by tenant, one-click rollback.
- Monitoring: Sentry for errors (with data scrubbing), OpenTelemetry traces from browser to API, Web Vitals tracking, uptime checks, and alerts on frontend error-rate spikes.
- Regions: per-region deployments for tenants with residency requirements; the CDN serves only static, non-tenant assets.

15. Project structure

```
frontend/
├── apps/
│   ├── web/                  # public site, docs, trust center
│   ├── console/              # tenant console
│   │   ├── app/
│   │   │   ├── (auth)/
│   │   │   └── (console)/
│   │   │       ├── overview/
│   │   │       ├── alerts/
│   │   │       ├── incidents/[id]/
│   │   │       ├── approvals/
│   │   │       ├── policy/
│   │   │       ├── sensors/
│   │   │       ├── reports/
│   │   │       ├── audit/
│   │   │       ├── settings/
│   │   │       └── billing/
```

```
|   |   └── tests/            # Playwright
|   └── operator/             # internal operator console
├── packages/
|   ├── ui/                   # shared design system (shadcn-based)
|   ├── api-client/           # generated OpenAPI client and Zod schemas
|   ├── explainability/       # reason, evidence, ATT&CK components
|   ├── realtime/             # WebSocket and SSE hooks
|   ├── rbac/                 # role and permission helpers
|   └── i18n/                 # en, am
└── .storybook/
```

A monorepo (pnpm workspaces with Turborepo or Nx) keeps the three apps consistent and shares the design system.

16. Delivery roadmap

Phase	Scope	Exit criteria
0. Foundations	Monorepo, design tokens, CI, auth shell, role-based layout, MSW mocks, OpenAPI client	Login, MFA and tenant routing work against mocks
1. Core loop	Alert queue, incident detail, explainability panel, audit log	An analyst can triage an alert end to end with evidence
2. Safety	Approvals, autonomy levels, kill switch, step-up auth	Dangerous actions are gated, reversible and audited
3. Onboarding	Setup wizard, sensor enrollment, detection-only mode, overview, sensor and model health	A new tenant reaches first real alert unaided
4. SaaS layer	Billing and usage, team management, SSO and SCIM, API keys, integrations	A paying tenant can self-manage
5. Trust and compliance	Reports, trust center, status page, security review, penetration test	Ready for a bank or government pilot
6. Operator console	Fleet, model rollout, support access	Platform team can run the service

Phase	Scope	Exit criteria
7. Advanced features	See section 17	Prioritized by pilot feedback

17. Advanced features (after the core loop is proven)

1. **AI investigation copilot:** natural-language questions about hosts, flows and timelines, with generated queries, cited evidence and incident summaries. It can propose actions but never execute them outside the approval flow.
2. **Detection-as-code:** policies and rules versioned, diffed, peer-reviewed and rolled back, with a "simulate against last week's traffic" preview before enabling.
3. **Replay and what-if:** run historical traffic through a candidate model version and compare decisions, tied to the shadow and canary pipeline.
4. **Case management:** shared investigations, assignment, notes, evidence timelines and shift handoff.
5. **Large-scale attack-path graph:** WebGL rendering with ATT&CK stage overlays and path highlighting.
6. **Keyboard-first triage:** command palette, shortcuts and saved views.
7. **Customer-facing executive reports:** scheduled, branded posture summaries.

These are differentiators, but they matter only once detection quality is proven. A polished interface on weak detection is a poor trade.

18. Open decisions and risks

Decisions needed

1. Is Amharic required in v1, or English only at launch?
2. Initial hosting: a single cloud region, or an Ethiopia-based region for government pilots?
3. Identity provider: managed (faster) or self-hosted (residency-friendly) for the first pilots?
4. One shared multi-tenant console with per-tenant subdomains, or separate deployments for government tenants?
5. Which SIEM integrations are required for the first pilot customer?

Risks

Risk	Mitigation
Tenant data leakage through UI, cache or WebSocket	Server-resolved tenant, isolation tests in CI, per-tenant channels, no shared client caches across sessions
Compromised control plane or operator console pushes harmful actions	Signed policy updates, tiered autonomy, human approval for destructive actions, customer kill switch, separate operator realm
Stored XSS from hostile traffic content	Strict escaping, CSP, fuzz tests with malicious payloads
Alert fatigue makes the product unusable	Grouping, suppression, load metrics, tuning workflows
Overstated AI claims damage credibility with regulated buyers	Honest wording in the UI and trust center; report precision, recall and false-alarm rates measured on real data
Scope creep into advanced features before the core loop works	Phase gates with exit criteria
Regulatory references out of date or wrong	Verify against primary sources before any customer-facing release

19. Next steps

1. Confirm the open decisions in section 18.
2. Define the API contract (OpenAPI) for alerts, incidents, approvals, policy and audit with the backend work.
3. Scaffold the monorepo and build Phase 0 against mocked data.
4. Design the alert queue and incident detail screens first, since they define the product's feel and value.